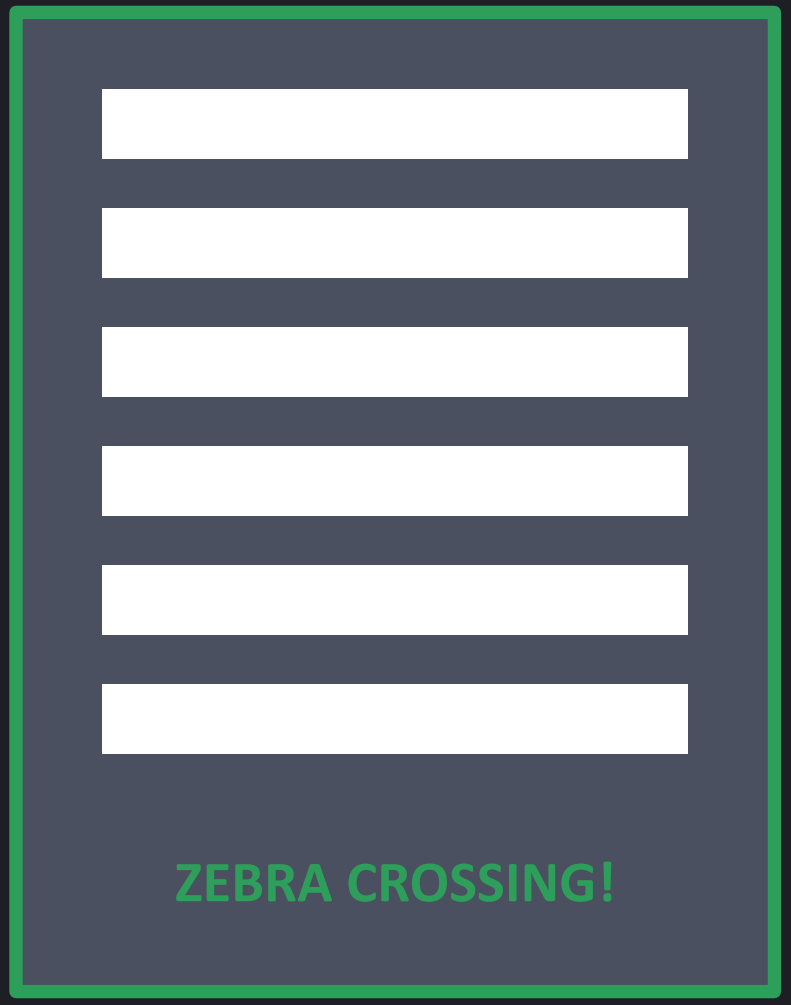
Real-Time Zebra-Crossing Detection on a Low-Cost FPGA Camera

A small 1-bit neural network that checks every camera picture, using under 0.3 W

Raj Mali · **Pooja Nagvekar**
Guides: Prof. Anuradha Joshi · Prof. Amaya Pethe
Vidyalankar Institute of Technology, Mumbai



Why detect zebra crossings?

Safety for blind and low-vision people

A walking aid can tell them a crossing is just ahead.

Driver assistance

A car can warn the driver or slow down near a crossing.

It must work on the spot

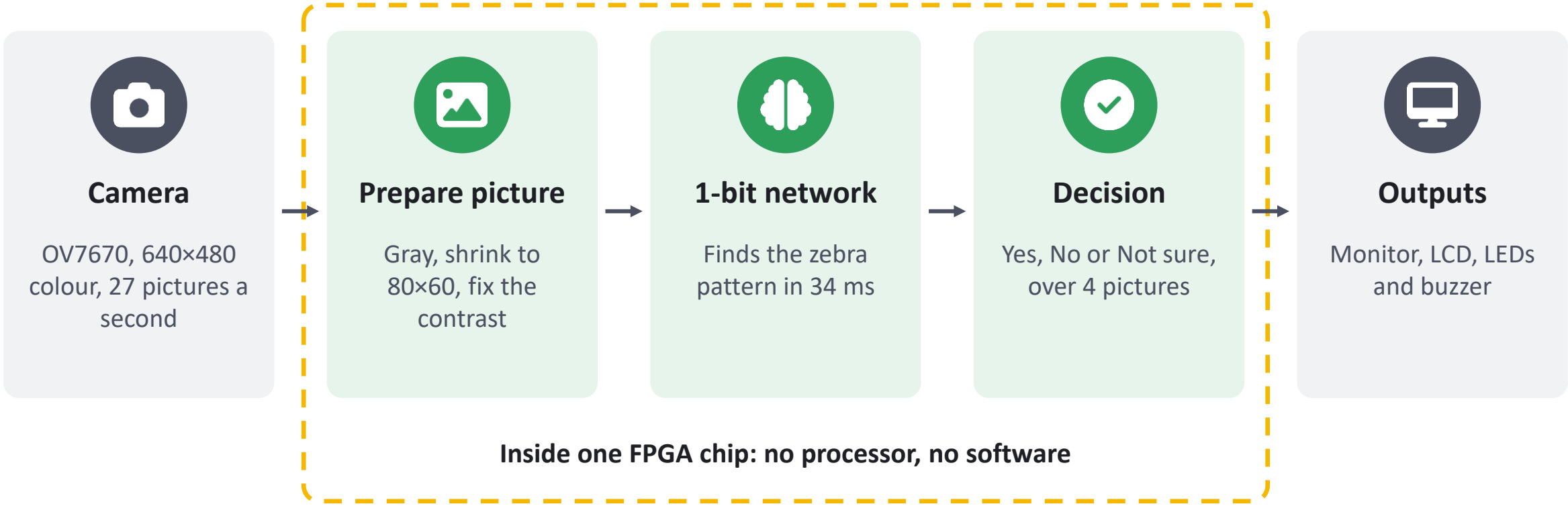
No internet, no big computer, a small battery, and a quick answer.

Our goal: spot zebra crossings live from a cheap camera, on one small low-power chip, with answers we can trust.



Street photo: Wikimedia Commons (CC0)

Our system at a glance



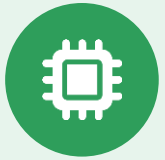
Hardware we used: EDGE Artix-7 FPGA board · OV7670 camera · VGA monitor · LCD, LEDs, buzzer · laptop (optional)

Why an FPGA, not a normal processor?



Normal processor (CPU)

- Runs software, one instruction after another
- Needs an operating system and memory chips
- Uses several watts; timing can vary



FPGA (what we used)

- We design the circuit itself
- Many parts work at the same time
- No software: the same speed every time
- Our whole design uses under 0.3 W



Think of it like this: a CPU is one cook following a recipe step by step. An FPGA is a kitchen we build for exactly one dish, where every station works at once. We describe the circuit in Verilog, a language for hardware: like drawing a circuit diagram in text.

Step 1: Preparing the picture



1. Camera picture

640×480, in colour



2. Gray, shrunk to 80×60

Only 4,800 pixels, but dull, like our camera's pictures



3. After auto-contrast

The stripes stand out clearly again

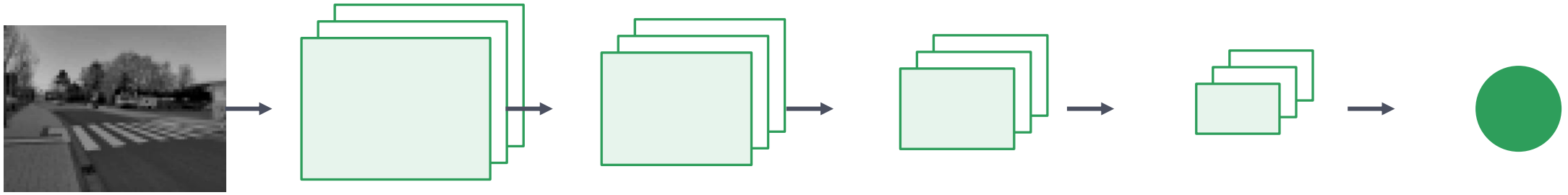
Why shrink?

The network stays small and fast. 80×60 still shows the stripes; 40×30 was much worse (61.7 % against 79.1 %).

Why auto-contrast?

Dull camera pictures dropped accuracy to 47.7 %. Stretching the brightness brings it back to about 78 %, in only 0.15 ms.

Step 2: The neural network



Picture	Layer 1	Layer 2	Layer 3	Layer 4	Score
80×60	32 maps, 40×30	64 maps, 20×15	64 maps, 10×7	64 maps, 5×3	1 number

Looks at small windows

Each layer slides a 3×3 window over the picture. Early layers find edges, later ones stripes, the last one “a crossing”.

Learned, not programmed

Nobody wrote a rule for “stripes”. The network learned the patterns from 11,000 example photos.

Shrinks step by step

Each layer halves the picture size, so the last layer sees the whole scene as a small 5×3 summary.

The key idea: a 1-bit network

Normal network: weights like 0.73 or −1.25, so every connection needs a multiplication: big, power-hungry circuits.
Our network: every weight and value is just +1 or −1, so each one is stored as a single bit.

Weights	**+1**	**−1**	**+1**	**+1**
Inputs	**+1**	**+1**	**−1**	**+1**
Same?	✓	✗	✗	✓

Normal maths: (+1)(+1) + (−1)(+1) + (+1)(−1) + (+1)(+1) = 0

1-bit way: 2 same − 2 different = 0 **same answer, no multiplier**

Multiply becomes “same or different?” (an XNOR gate). Adding becomes counting.

0

multipliers needed for the network

11 KB

all 93,408 weights fit inside the chip

64

values compared in one clock tick

Step 3: How the chip runs the network

One engine, used again and again

The same small circuit runs all five layers, one after another.

64 checks every clock tick

Each tick it compares 64 values at once and counts the matches.

Everything stays on the chip

Picture, weights and results sit in the chip's own memory.

2.27 million ticks per picture

At 66.7 million ticks per second, that is 34 ms.

Fast enough for every picture

A new camera picture arrives every

37.5 ms

Our network needs

34.0 ms

Finished before the next picture comes

26.7

answers every second

16 %

of the FPGA chip used

Training the network (on a laptop)

1

Collect photos

11,000 pictures from 14 sources: dashcams, streets, CCTV. Test photos are kept apart and never used in training.

2

Train

In Python (PyTorch), 60 rounds. A big “teacher” network (ResNet-18) guides our small one.

3

Convert to bits

Each weight becomes 1 bit. Each neuron's settings become one number: its threshold.

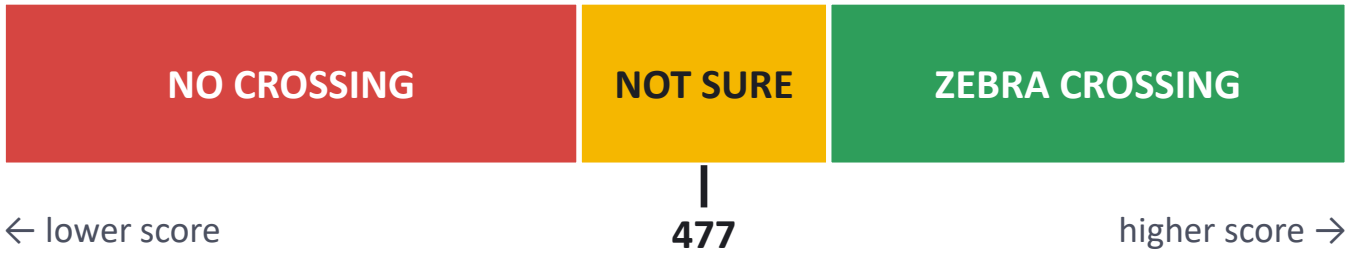
4

Load into the FPGA

The same files fill the chip's memory and set up the circuit, so they always match.

Training trick: a 1-bit value cannot learn in small steps. So while training we keep a hidden normal number for each weight and use only its sign (+ or −). Learning passes through the sign step as if it were not there: the “straight-through estimator”.

Step 4: The decision, honest about doubt



The network gives a score from 0 to 960. Above 477 means crossing; below means no crossing.

Too close to 477? It says NOT SURE instead of guessing.

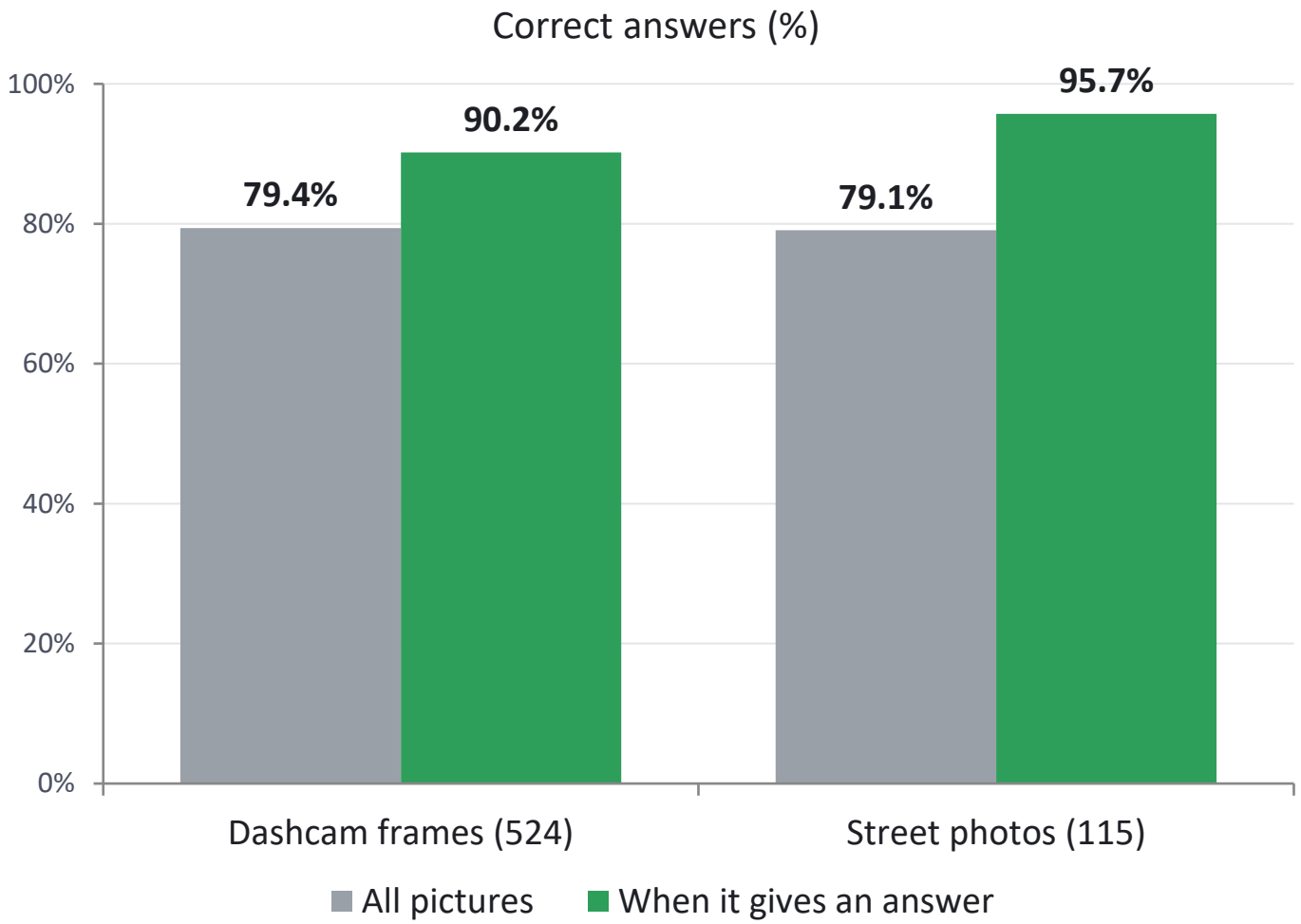
Average of the last 4 pictures: a steady answer that does not flicker.

Shown on: monitor border colour, LCD text, LEDs and a buzzer beep.



What the monitor shows: green border, the decision, a score bar, and the 80×60 picture the network sees (top right). Frame from our design's simulation.

Results: how accurate is it?



4 in 5

pictures correct overall (about 79 %)

19 in 20

street photos correct when it answers (95.7 %)

Test pictures were never seen in training. For the rest of the pictures it honestly says “not sure” (about half of them).

Proof: the chip gives exactly the same answers

639 / 639

test pictures gave identical answers on the real FPGA board and in our Python model: bit for bit.

Board self-test at power-up: ALL PASS

Python model

A software copy of the chip's exact maths, with the same bits and integers.

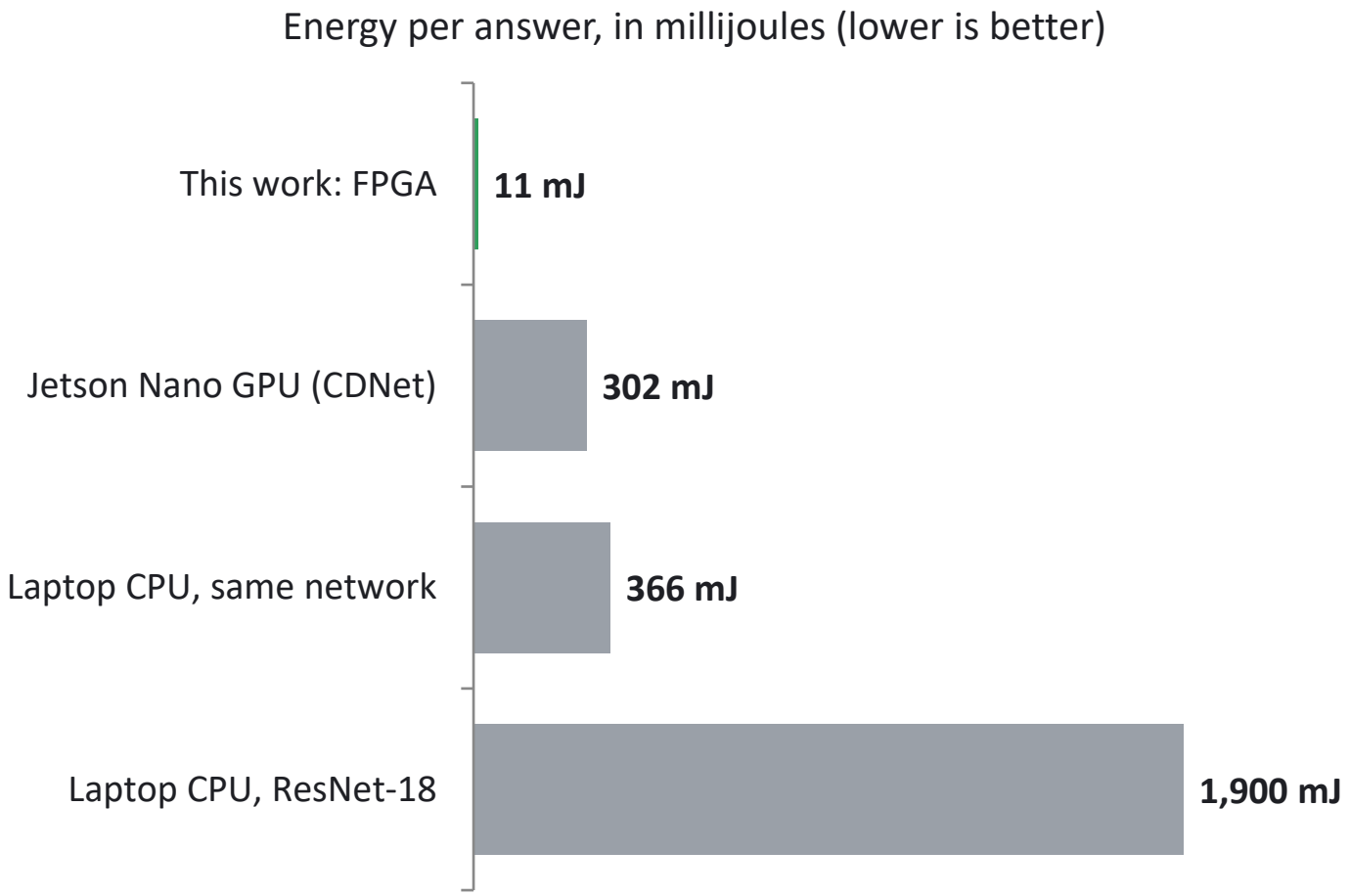
Circuit simulation

Every value inside every layer matched, not just the final answer.

Real FPGA board

Pictures sent from the laptop over USB; every answer compared one by one.

Comparison: much less energy per answer



Energy per answer = power × time for one answer.
FPGA: Vivado estimate · Jetson Nano: CDNet paper · Laptop: i5-12500H, 45 W × measured time

27× less

energy than a Jetson Nano GPU board

0.29 W

total power of our FPGA design (Jetson Nano: 10 W)

1 chip

camera to decision: no PC, no GPU, no internet

Limitations and future work



Limitations

- About 1 in 5 hard street photos are still wrong
- 1-bit weights cost accuracy: a full-size network (ResNet-18) gets 95.7 % on the same small pictures
- Night, glare and unusual angles are the hardest
- It says yes or no, not where the crossing is



Next steps

- Train with real pictures from our own camera
- Show where the crossing is in the picture
- A bigger FPGA for a wider, more accurate network
- A custom chip (ASIC) for battery-powered walking aids

Conclusion

34 ms

Real time

Every camera picture is checked

0.29 W

Low power

27× less energy than a Jetson Nano

639/639

Proven

Identical answers on the real board

Thank you. Questions?

Raj Mali · Pooja Nagvekar | Guides: Prof. Anuradha Joshi · Prof. Amaya Pethe | Vidyalankar Institute of Technology, Mumbai